



Contents

7.1	The List ADT	258
7.2	Array Lists	260
7.2.1	Dynamic Arrays	263
7.2.2	Implementing a Dynamic Array	264
7.2.3	Amortized Analysis of Dynamic Arrays	265
7.2.4	Java's StringBuilder class	269
7.3	Positional Lists	270
7.3.1	Positions	272
7.3.2	The Positional List Abstract Data Type	272
7.3.3	Doubly Linked List Implementation	276
7.4	Iterators	282
7.4.1	The Iterable Interface and Java's For-Each Loop	283
7.4.2	Implementing Iterators	284
7.5	The Java Collections Framework	288
7.5.1	List Iterators in Java	289
7.5.2	Comparison to Our Positional List ADT	290
7.5.3	List-Based Algorithms in the Java Collections Framework	291
7.6	Sorting a Positional List	293
7.7	Case Study: Maintaining Access Frequencies	294
7.7.1	Using a Sorted List	294
7.7.2	Using a List with the Move-to-Front Heuristic	297
7.8	Exercises	300

7.1 The List ADT

In Chapter 6, we introduced the stack, queue, and deque abstract data types, and discussed how either an array or a linked list could be used for storage in an efficient concrete implementation of each. Each of those ADTs represents a linearly ordered sequence of elements. The deque is the most general of the three, yet even so, it only allows insertions and deletions at the front or back of a sequence.

In this chapter, we explore several abstract data types that represent a linear sequence of elements, but with more general support for adding or removing elements at arbitrary positions. However, designing a single abstraction that is well suited for efficient implementation with either an array or a linked list is challenging, given the very different nature of these two fundamental data structures.

Locations within an array are easily described with an integer *index*. Recall that an index of an element e in a sequence is equal to the number of elements before e in that sequence. By this definition, the first element of a sequence has index 0, and the last has index $n - 1$, assuming that n denotes the total number of elements. The notion of an element's index is well defined for a linked list as well, although we will see that it is not as convenient of a notion, as there is no way to efficiently access an element at a given index without traversing a portion of the linked list that depends upon the magnitude of the index.

With that said, Java defines a general interface, `java.util.List`, that includes the following index-based methods (and more):

- `size()`: Returns the number of elements in the list.
- `isEmpty()`: Returns a boolean indicating whether the list is empty.
- `get( $i$ )`: Returns the element of the list having index i ; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.
- `set( $i, e$ )`: Replaces the element at index i with e , and returns the old element that was replaced; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.
- `add( $i, e$ )`: Inserts a new element e into the list so that it has index i , moving all subsequent elements one index later in the list; an error condition occurs if i is not in range $[0, \text{size}())]$.
- `remove( $i$ )`: Removes and returns the element at index i , moving all subsequent elements one index earlier in the list; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.

We note that the index of an existing element may change over time, as other elements are added or removed in front of it. We also draw attention to the fact that the range of valid indices for the `add` method includes the current size of the list, in which case the new element becomes the last.

Example 7.1 demonstrates a series of operations on a list instance, and Code Fragment 7.1 below provides a formal definition of our simplified version of the List interface; we use an `IndexOutOfBoundsException` to signal an invalid index argument.

Example 7.1: *We demonstrate operations on an initially empty list of characters.*

Method	Return Value	List Contents
add(0, A)	–	(A)
add(0, B)	–	(B, A)
get(1)	A	(B, A)
set(2, C)	“error”	(B, A)
add(2, C)	–	(B, A, C)
add(4, D)	“error”	(B, A, C)
remove(1)	A	(B, C)
add(1, D)	–	(B, D, C)
add(1, E)	–	(B, E, D, C)
get(4)	“error”	(B, E, D, C)
add(4, F)	–	(B, E, D, C, F)
set(2, G)	D	(B, E, G, C, F)
get(2)	G	(B, E, G, C, F)

```

1  /** A simplified version of the java.util.List interface. */
2  public interface List<E> {
3      /** Returns the number of elements in this list. */
4      int size();
5
6      /** Returns whether the list is empty. */
7      boolean isEmpty();
8
9      /** Returns (but does not remove) the element at index i. */
10     E get(int i) throws IndexOutOfBoundsException;
11
12     /** Replaces the element at index i with e, and returns the replaced element. */
13     E set(int i, E e) throws IndexOutOfBoundsException;
14
15     /** Inserts element e to be at index i, shifting all subsequent elements later. */
16     void add(int i, E e) throws IndexOutOfBoundsException;
17
18     /** Removes/returns the element at index i, shifting subsequent elements earlier. */
19     E remove(int i) throws IndexOutOfBoundsException;
20 }

```

Code Fragment 7.1: A simple version of the List interface.